

Projet Bases de Données ArtConnect

TI603 - Bases de données 2

17 mai 2026

Sommaire

I - Conception et l'implémentation de la base de données

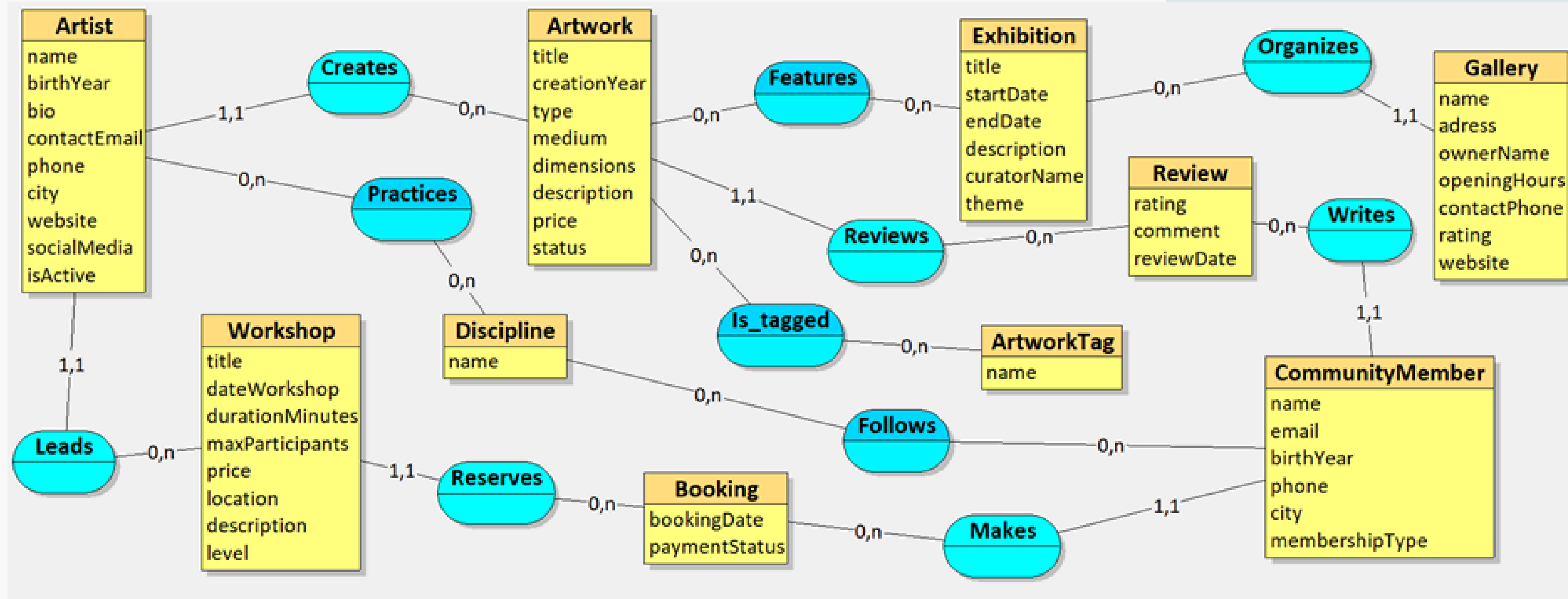
II - Objets clés de la base de données

III - Architecture de l'application

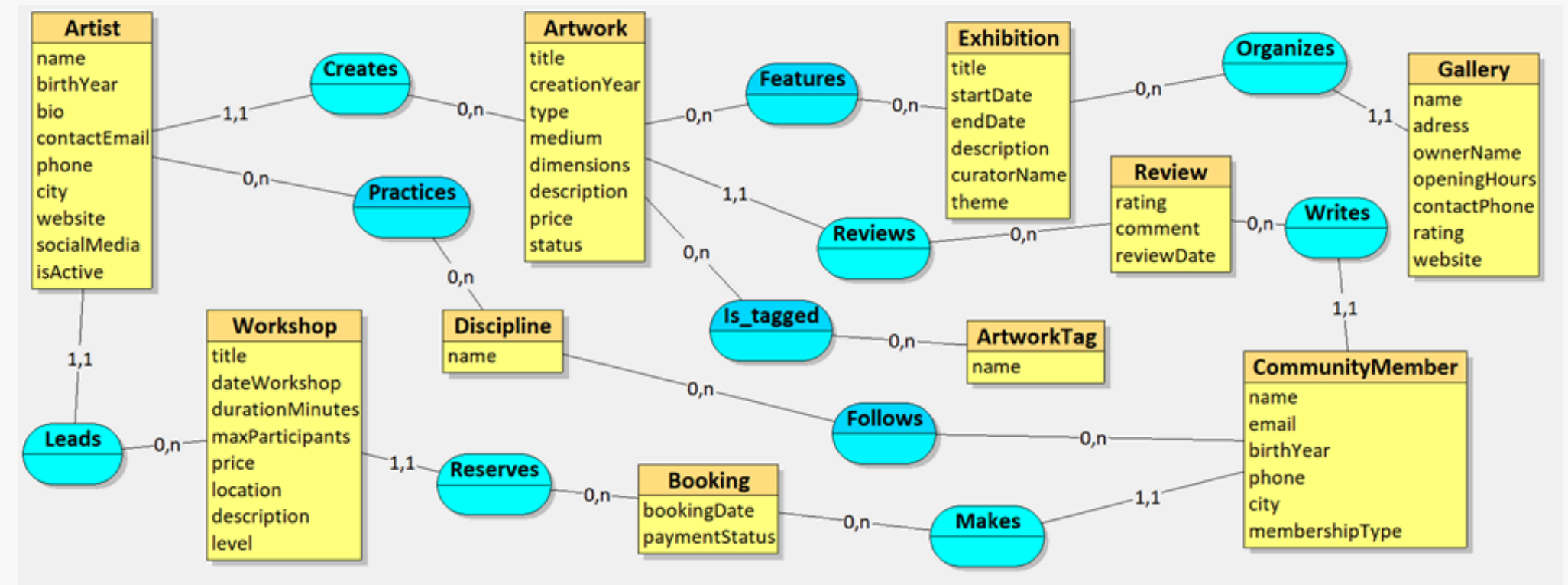
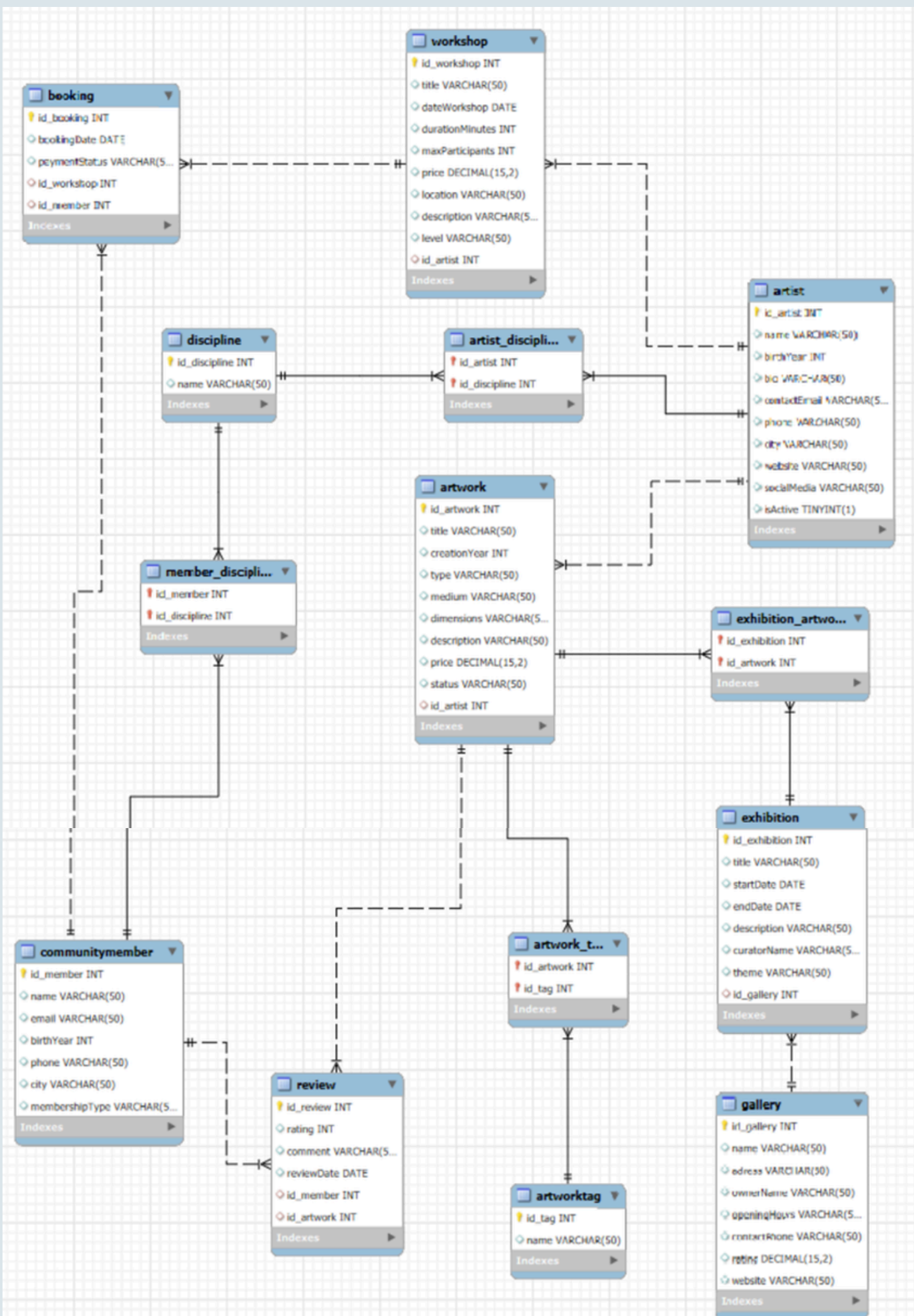
IV - Les défis rencontrés et les solutions apportées

V - Démonstration de l'application avec la base de données

Modèle conceptuel des données (MCD)



Modèle logique des données (MLD)



Bases de données - Vues

VUE 1 : v_artist_overview

Objectif : Vue de synthèse publique sur les artistes actifs en masquant les données sensibles pour une question de sécurité.

VUE 2 : v_artwork_catalog

Objectif : Catalogue complet des œuvres avec artiste et tags.

VUE 3 : v_exhibition_program

Objectif : Programme des expositions avec galerie et oeuvres exposés

VUE 5 : v_member_activity

Objectif : Résumé de l'activité de chaque membre avec le nombre d'ateliers suivis, nombre avis, montant total dépensé.

VUE 4 : v_workshop_availability

Objectif : Ateliers avec places restantes calculées en temps réel

Bases de données - Index

Index 1 : idx_artwork_status

Objectif : Optimiser le filtrage des œuvres par statut (FOR_SALE, SOLD, EXHIBITED) pour éviter un parcours complet de la table.

Index 2 : idx_artwork_artist

Objectif : Optimiser la récupération de toutes les œuvres d'un artiste donné via la clé étrangère id_artist.

Index 3 : idx_booking_workshop

Objectif : Optimiser le comptage et la liste des inscrits à un atelier. utilisé notamment par le trigger trg_booking_check_capacity.

Index 4 : idx_exhibition_dates

Objectif : Optimiser la recherche des expositions en cours ou à venir en filtrant sur startDate et endDate.

Objectif : Les index optimisent les requêtes les plus fréquentes d'ArtConnect, filtrage des œuvres, vérification de capacité des ateliers et recherche d'expositions par dates, afin de garantir des temps de réponse rapides.

Bases de données - Déclencheurs

TRIGGER 1 : trg_booking_check_capacity

S'active avant l'insertion d'une réservation.

Ce trigger permet d'empêcher de dépasser la capacité maximale d'un atelier.

TRIGGER 3 : trg_review_rating_range

S'active avant l'insertion d'un avis.

Permet de contraindre la note entre 1 et 5 étoiles.

TRIGGER 2 : trg_exhibition_date_check

S'active avant l'insertion et mise à jour d'une exposition.

Ce trigger permet de garantir que la date de fin est toujours postérieure à la date de début.

Bases de données - Procédures et Fonctions

Fonction 1 : `get_participants_count`

Objectif : Permettre de connaître le nombre de participants inscrits à un atelier, sans avoir à écrire la requête côté application.

Procédure 2 : `create_workshop`

Objectif : Créer un nouvel atelier en passant directement tous ses attributs en paramètres pour garantir une insertion cohérente et éviter les erreurs de saisie.

Procédure 3 : `get_artists_by_city`

Objectif : Récupérer tous les artistes actifs d'une ville donnée en paramètre avec leurs disciplines associées.

Bases de données - Transaction

Procédure utilisée : `sp_register_for_art_path`

Objectif : Inscrire un membre à deux ateliers en une seule opération.

Scenario

Un membre s'inscrit à deux ateliers en même temps. Les deux inscriptions doivent réussir ensemble ou être annulées ensemble.

Cas ECHEC :

Si un atelier est complet, une erreur est déclenchée et toutes les inscriptions sont annulées automatiquement grâce au ROLLBACK. Aucune réservation n'est enregistrée

Cas SUCCES :

Les deux INSERT dans Booking s'exécutent correctement. Le COMMIT valide les deux inscriptions. Un message de succès apparaît

Architecture de l'application

1. Modèle (Entities) : Objets Java simples représentant les données (Artist, Artwork, etc.).

3. Service : Centralise la logique applicative et fait le pont entre l'interface et les données via le ServiceProvider.

2. Présentation (UI) : Interfaces graphiques JavaFX (fichiers FXML) et leurs Controllers qui gèrent les clics et l'affichage.

4. Accès aux Données (DAO/Persistence) : Gère les requêtes SQL via JDBC pour communiquer avec la base de données MySQL.

Utilisateur → Controller → Service → DAO (JDBC) → MySQL

Défis rencontrés et solutions apportées

1. Compréhension DAO et JDBC

Difficulté : Comprendre le fonctionnement de ces fichiers

Solution : On a pris le temps de lire le code et les exemples fournis et on a implémenté chaque DAO progressivement en testant à chaque étape.

2. Création des fenêtres de saisie

Difficulté : L'interface ne proposait pas de formulaire pour ajouter un artiste ou une œuvre.

Solution : On a créé des fenêtres popup JavaFX avec tous les champs nécessaires et une validation des données saisies.

3. Bouton de suppression malfunctionnel :

Difficulté : Le bouton Delete supprimait les objets créés via l'interface mais pas ceux déjà présents en base.

Solution : On a ajouté ON DELETE CASCADE sur toutes les clés étrangères pour que la suppression fonctionne même quand un enregistrement est lié à d'autres tables.

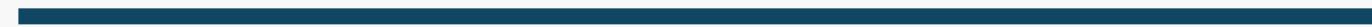
.....

Démonstration de l'application

.....



Merci !



LIM Tommy

LAI Ivan

KAAWAR Erwan